# Exploit Analysis using Python

Tushar B. Kute,
http://tusharkute.com

# Exploit

- Definition:
  - A specific tool or method used to abuse a security flaw.
- Function:
  - Acts as a "delivery vehicle" for malicious payloads like ransomware or spyware.
- Goal:
  - To bypass security controls, steal sensitive data, or take full control of a system.
- Key Distinction:
  - An exploit is the action; a vulnerability is the condition.

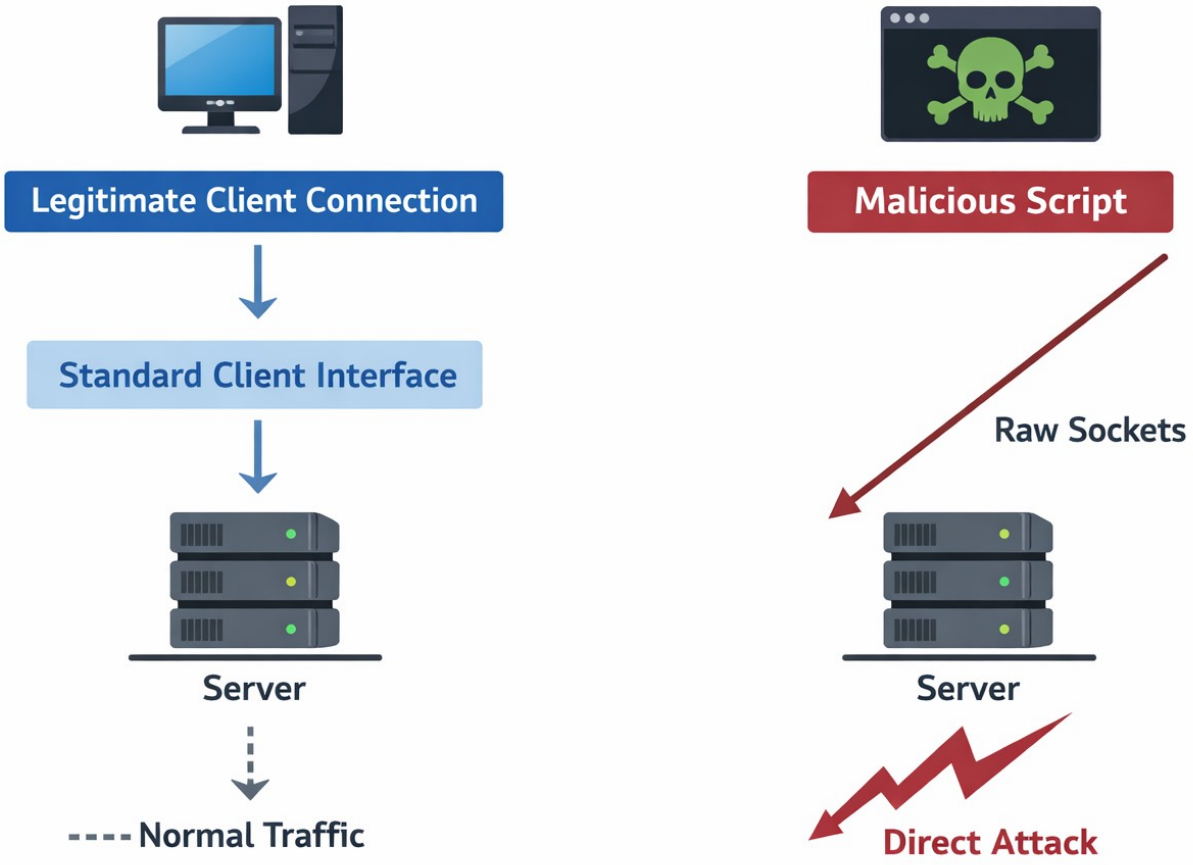
# What is a Socket?

- A software endpoint for two-way communication between programs.
- Facilitates data transfer across networks (TCP/UDP).
- Crucial Concept:
  - Sockets are just "pipes." They are inherently neither secure nor insecure; they simply transport bytes.

# Where Do Flaws Originate?

- Vulnerabilities rarely exist in the networking protocols (TCP/IP) themselves.
- Flaws exist in how the application processes the data received from the socket.
- Rule #1:
  - All data read from socket.recv() must be treated as hostile.

# Sockets from an Attacker's Perspective



# Sockets from an Attacker's Perspective

- Security researchers and malicious actors use custom socket scripts to probe applications.
- Sockets allow precise, low-level control over exactly what bytes are sent, bypassing client-side validation (like web browser limits).

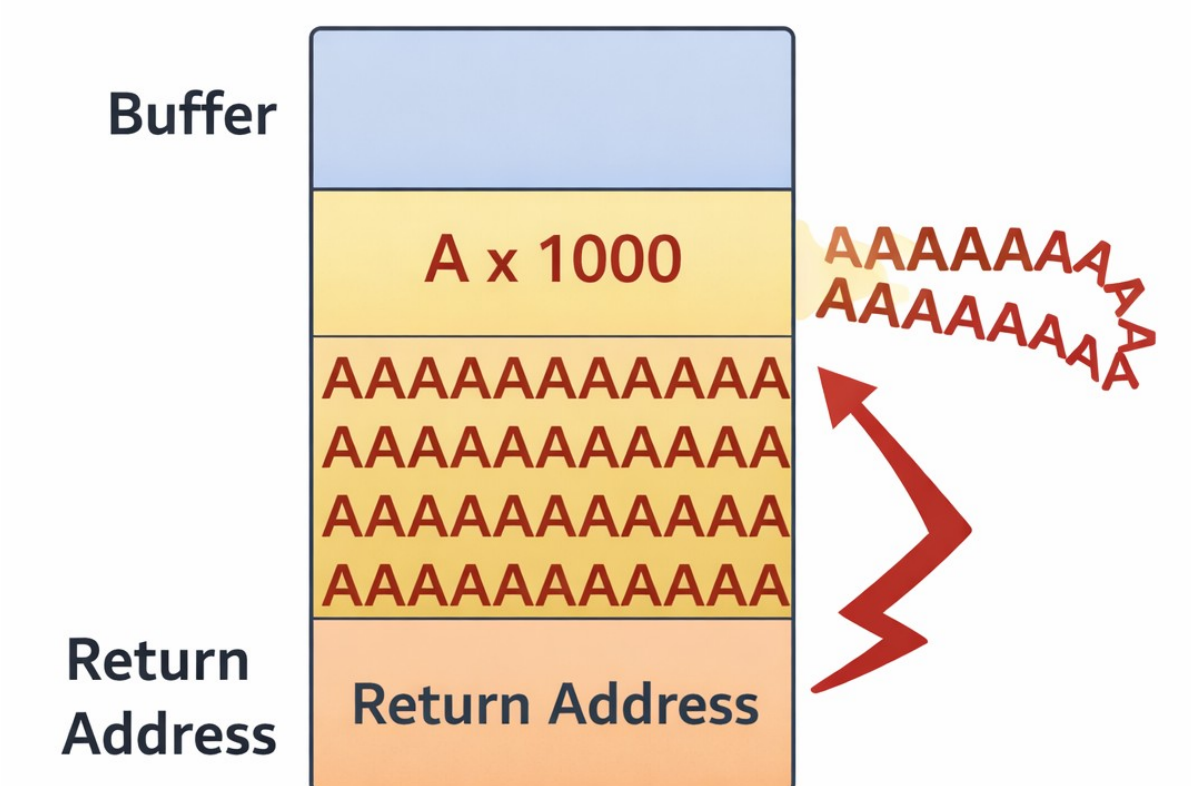
# Three Primary Threat Vectors

- Memory Safety Flaws (Buffer Overflows)
- Injection Flaws (Command Injection)
- Availability Flaws (Denial of Service - DoS)

# Understanding Buffer Overflows

- Common in applications written in C/C++.
- Occurs when a program allocates a fixed chunk of memory (a buffer) but accepts more data than it can hold.
- Excess data spills over, overwriting adjacent memory regions.

# The Anatomy of an Overflow



# The Anatomy of an Overflow

- If an attacker carefully crafts the overflowing data, they can overwrite the program's return address.
- This allows them to redirect the program's execution flow to their own malicious code (shellcode).

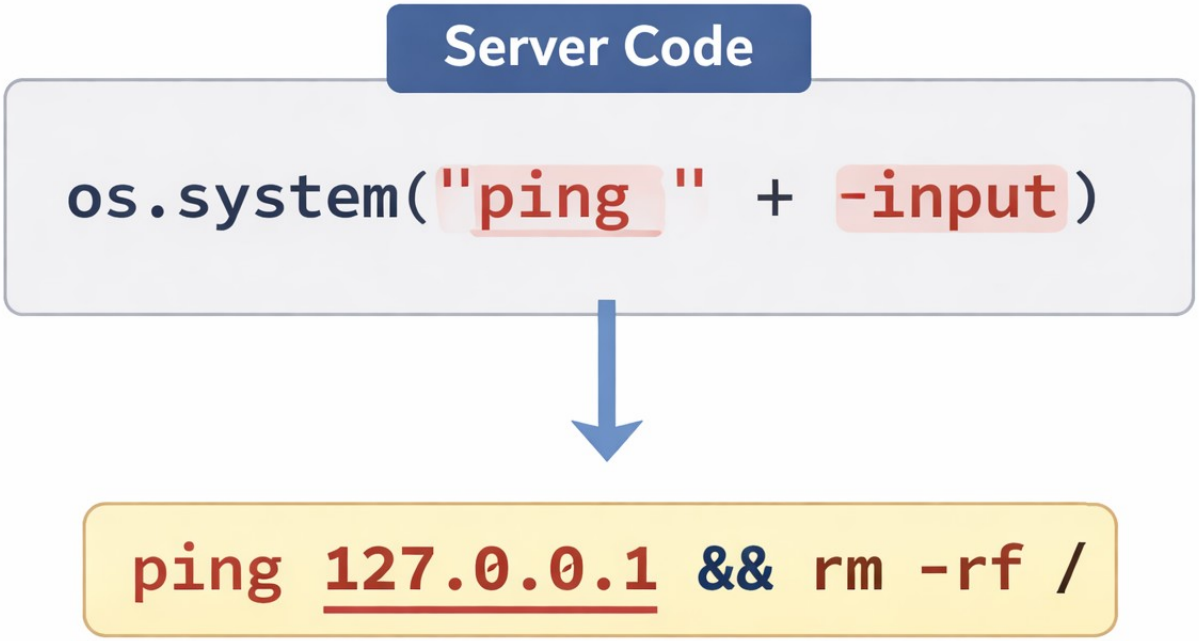
# Delivering the Payload

- A server listens on a socket and expects a 50-byte username.
- The attacker writes a Python script using socket.sendall() to send 5,000 bytes.
- If the server uses unsafe C functions (like strcpy) on the socket data, the application crashes or is compromised.

# What is Command Injection?

- Occurs when an application passes unsafe user-supplied data directly to a system shell.
- The application intends to run one command, but the attacker appends additional commands.

# The Injection Vector



# The Injection Vector

- A network service takes an IP address via a socket to execute a ping.
- Expected input: 8.8.8.8
- Attacker sends: 8.8.8.8 ; cat /etc/shadow
- The server executes both, exposing sensitive system files.

# Denial of Service (DoS)

- The goal is not to steal data, but to make the server unavailable to legitimate users.
- Achieved by exhausting server resources (CPU, Memory, Network Bandwidth, or File Descriptors/Ports).

# Exhausting Sockets

- Servers have a limit on concurrent open sockets.
- Attacker script opens thousands of TCP connections but never sends data and never closes them.
- The server reaches its limit and drops real user connections.

# Crashing the Parser

- Servers parse incoming socket data (e.g., reading JSON, XML, or custom headers).
- Attackers send intentionally malformed, incomplete, or recursive data over the socket.
- The parsing logic gets stuck in an infinite loop or throws an unhandled exception, crashing the server.

# Defensive Socket Programming

- Knowing how attacks work informs how we build resilient systems.
- The overarching principle: Zero Trust.

# Defence-1 Input Validation & Sanitization

- Validate the length:
  - Ensure recv() data does not exceed expected limits.
- Validate the type/format:
  - Use Regular Expressions (Regex) to ensure data matches strict expected patterns.
- Reject, do not try to fix, bad data.

# Secure Parsing Logic

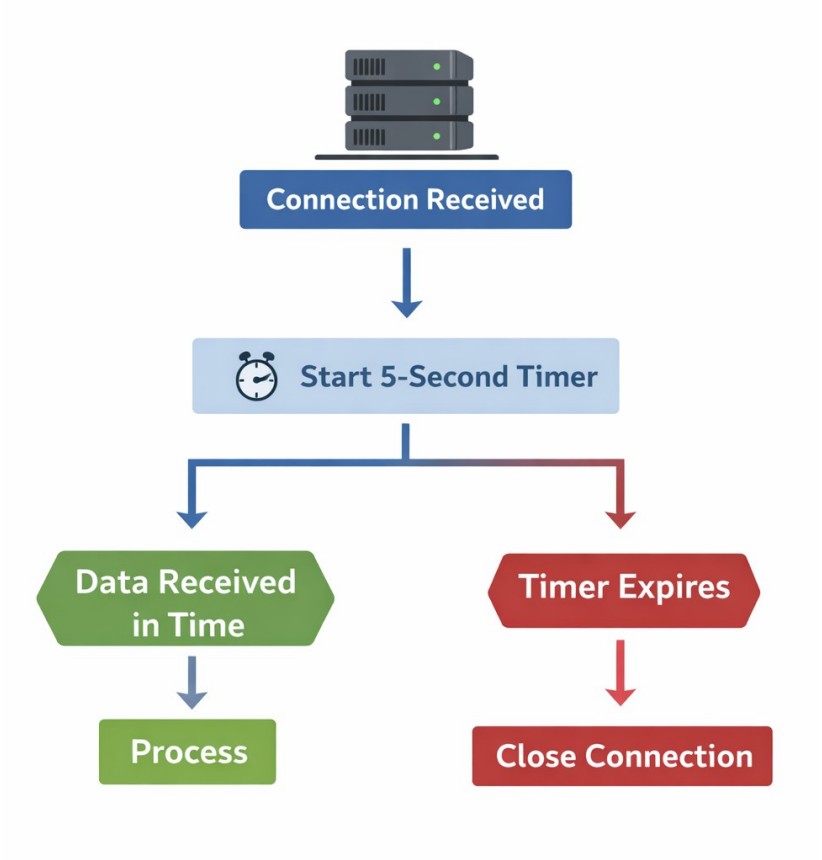
- Show how to safely process socket data.

```
data = conn.recv(1024).decode().strip()
if not re.match(r"^[a-zA-Z0-9]+$", data):
    conn.close() # Drop immediately
```

# Defense 2 - Resource Management Implementing Timeouts

- Never let a socket block forever on accept() or recv().
- Use socket.settimeout() to enforce strict communication windows.
- Drops slow or dead connections automatically.

# Safely Handling Exceptions



# Safely Handling Exceptions

- Network code must be wrapped in try...except blocks.
- Handle socket.timeout and ConnectionResetError gracefully without crashing the main server loop.

# Defense 3 - In-Transit Security The Danger of Plaintext

# Defense 3 - In-Transit Security

- Standard TCP/UDP sockets send data in cleartext.
- Vulnerable to Packet Sniffing and Man-in-the-Middle (MitM) attacks.
- An attacker on the same network can read or modify the socket traffic.

# Wrapping Sockets in TLS/SSL

- TLS (Transport Layer Security) encrypts the byte stream.
- Guarantees confidentiality and data integrity.
- In Python, this is achieved using the built-in ssl module.

# Summary

- Assume all data from recv() is an attack.
- Validate format, length, and content strictly.
- Use timeouts to prevent resource exhaustion.
- Wrap sensitive sockets in TLS encryption.

# Thank you



@mitu_skillologies

@mITuSkillologies

@mitu_group

@mitu-skillologies

@MITUSkillologies

kaggle

@mituskillologies

**Web Resources**
https://mitu.co.in
http://tusharkute.com

@mituskillologies

**contact@mitu.co.in**

**tushar@tusharkute.com**

# Public Feedback